PROJECT REPORT

B.Tech CSE (III YEAR – V SEM) (2024–25)

DEPARTMENT OF COMPUTER ENGINEERING

GLA UNIVERSITY - 17km Stone, NH-19 (or NH-2), Mathura-Delhi Road, P.O. Chaumuhan, Mathura-281 406, Uttar Pradesh, India

PROJECT TITLE: AI-BASED OPHTHALMOLOGY DIAGNOSTIC TOOL (EYEINTEL)

Major Project (CSE)

Report Submission Date: 4 November 2025

Team Name: Vision Seekers (T24)

Team Leader: Durgesh Kumari Tarkar
Team Member 1: Deepika Kumari
Team Member 2: Boby Sharma

Project Mentor: Mr. Shivanshu Upadhyay

Mentor Signature: _____

Index

S. No.	Content	Page No.
1	Project Overview & Working	2
2	Week-wise Timeline	3
3	Individual Contributions	4
4	Technical Documentation & Research Work	5
5	Snapshots	6–7

Project Overview & Working

The AI-Based Ophthalmology Diagnostic Tool (EyeIntel) is a deep learning-based system designed to assist in the early detection of eye diseases such as Cataract, Glaucoma, and Diabetic Retinopathy. By analyzing retinal images, the system can classify the disease and provide a confidence score, helping ophthalmologists and healthcare providers make informed decisions.

The project integrates a Flask backend with a trained CNN model for image classification. Frontend interfaces are developed by Deepika Kumari using HTML, CSS, and JavaScript, providing users with an intuitive experience for uploading images and viewing diagnostic results. The generated PDF report contains disease prediction and precautionary advice.

Week-wise Project Timeline

Week	Task Description	Assigned To	Status
1	Dataset collection and Flask setup	All Members	Completed
2	Image preprocessing pipeline & UI wireframes	Deepika	Completed
3	CNN model training & API creation	Durgesh	Completed
4	Frontend-backend integration	All Members	Completed
5	Model evaluation & dashboard creation	Durgesh	Completed
6	Testing and debugging	All Members	Completed
7	Deployment and optimization	Boby	In Progress
8	Documentation & Final Demo	All Members	Completed

Individual Contributions

Durgesh Kumari Tarkar (Team Leader)
• Designed and trained the CNN model for retinal image classification.
• Implemented model integration with the Flask backend.
• Led backend development, database setup, and report generation module.
GitHub Branch: Durgesh

Deepika Kumari
• Designed and developed the entire frontend using HTML, CSS, and JavaScript.
• Focused on responsive design and a user-friendly interface.
• Contributed to UI-backend communication and application testing.
GitHub Branch: Deepika

Boby Sharma
• Responsible for project deployment and server configuration.
• Worked on app hosting, optimization, and environment debugging.
• Deployment is currently in progress.
GitHub Branch: Boby

Technical Documentation & Research Work

Backend (All Members): Developed using Flask, integrated with TensorFlow for AI model predictions, and SQLAlchemy for database handling.

Frontend (Deepika): Designed using HTML, CSS, and JavaScript ensuring smooth UI/UX.

AI Model (Durgesh): CNN-based deep learning model trained on retinal image datasets.

Deployment (Boby): Managed server setup and integration; ongoing final optimization.

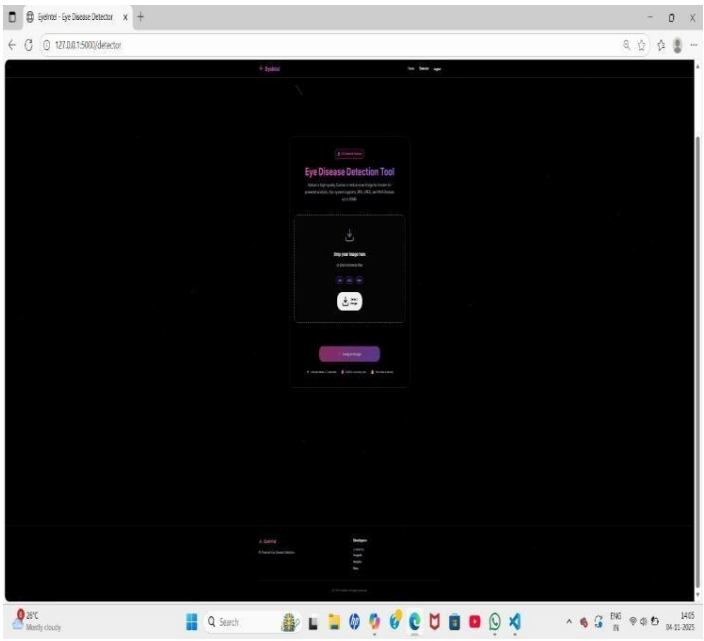
Database (All Members): SQLite database for user authentication and image tracking.

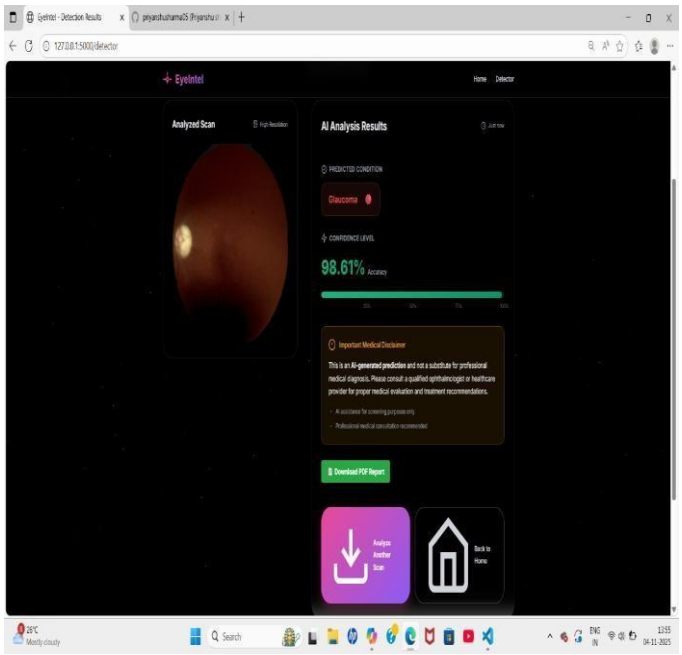
Research Work: Comparative study of existing ophthalmology AI systems like IDx-DR and EyeArt, focusing on accessibility and early disease detection.

Snapshots

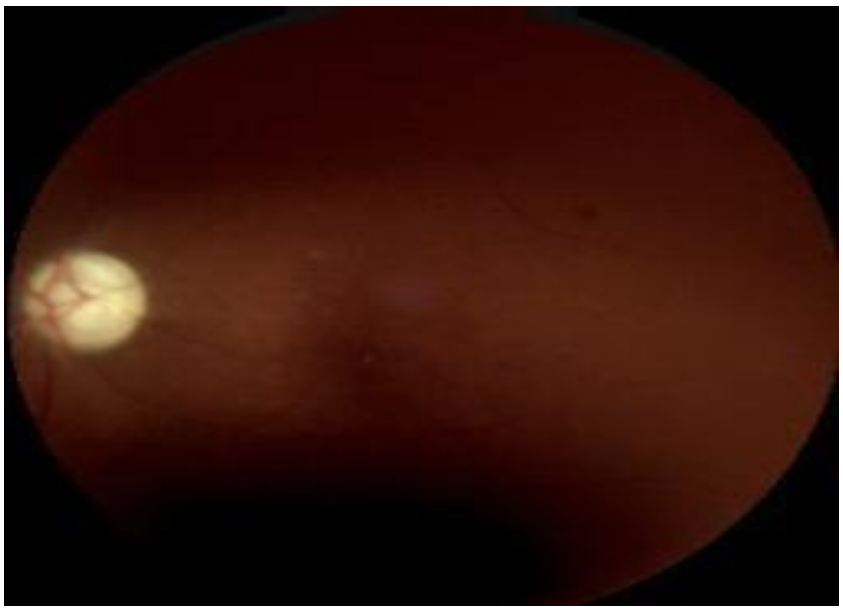
1. Home Page

2. Detector Page

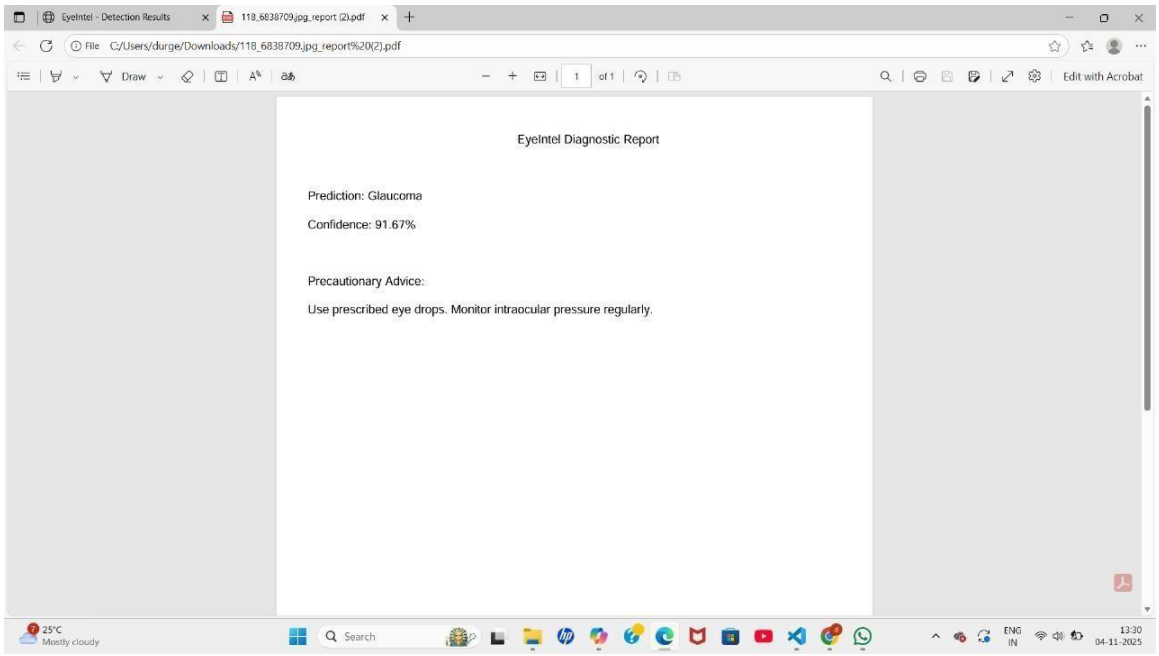




3. Uploaded Image Preview



4. Generated PDF Report



5. app.py

```python
import os
import io
import numpy as np
from PIL import Image
from fpdf import FPDF
from flask import Flask, request, render_template, url_for, redirect, flash
from flask_sqlalchemy import SQLAlchemy
from flask_login import LoginManager, login_user, login_required, logout_user, UserMixin
from werkzeug.utils import secure_filename
from werkzeug.security import generate_password_hash, check_password_hash
import tensorflow as tf

# --- Initialization ---
app = Flask(__name__)
app.secret_key = 'your_secret_key'  # Replace with a secure key
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///users.db'
app.config['UPLOAD_FOLDER'] = 'static/uploads/'
ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg'}
os.makedirs(app.config['UPLOAD_FOLDER'], exist_ok=True)

# --- Database & Login Setup ---
db = SQLAlchemy(app)
login_manager = LoginManager()
login_manager.init_app(app)
login_manager.login_view = 'login'

# --- User Model ---
class User(UserMixin, db.Model):
    id = db.Column(db.Integer, primary_key=True)
```

```python
# --- User Model ---
class User(UserMixin, db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(150), unique=True, nullable=False)
    password = db.Column(db.String(150), nullable=False)

@login_manager.user_loader
def load_user(user_id):
    return User.query.get(int(user_id))

# --- Load Model ---
try:
    MODEL_PATH = 'model/my_model.h5'  # Updated to match your actual file
    model = tf.keras.models.load_model(MODEL_PATH)
    print("✅ Model loaded successfully.")
except Exception as e:
    print(f"❌ Error loading model: {e}")
    model = None

CLASS_NAMES = ['cataract', 'diabetic_retinopathy', 'glaucoma', 'normal']

# --- Helper Functions ---
def allowed_file(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS

def preprocess_image_for_prediction(image_bytes):
    img = Image.open(io.BytesIO(image_bytes)).convert('RGB')
    img = img.resize((256, 256))  # Resize if your model expects fixed input
    img_array = tf.keras.preprocessing.image.img_to_array(img)
```

```python
def preprocess_image_for_prediction(image_bytes):
    img = Image.open(io.BytesIO(image_bytes)).convert('RGB')
    img = img.resize((256, 256))  # Resize if your model expects fixed input
    img_array = tf.keras.preprocessing.image.img_to_array(img)
    img_array = tf.expand_dims(img_array, 0)
    return img_array

def predict_disease(image_bytes):
    if model is None:
        return "Error", "Model not loaded. Please check the server logs."
    processed_image = preprocess_image_for_prediction(image_bytes)
    predictions = model.predict(processed_image)
    predicted_class_index = np.argmax(predictions[0])
    predicted_class = CLASS_NAMES[predicted_class_index]
    confidence = round(100 * np.max(predictions[0]), 2)
    return predicted_class, confidence

def generate_pdf(predicted_class, confidence, filename):
    pdf = FPDF()
    pdf.add_page()
    pdf.set_font("Arial", size=12)

    pdf.cell(200, 10, txt="EyeIntel Diagnostic Report", ln=True, align='C')
    pdf.ln(10)
    pdf.cell(200, 10, txt=f"Prediction: {predicted_class}", ln=True)
    pdf.cell(200, 10, txt=f"Confidence: {confidence}%", ln=True)
    pdf.ln(10)

    precautions = {
        "Cataract": "Consult an ophthalmologist for possible surgery. Avoid driving at night.",
```

```python
def generate_pdf(predicted_class, confidence, filename):
    precautions = {
        "Cataract": "Consult an ophthalmologist for possible surgery. Avoid driving at night.",
        "Diabetic Retinopathy": "Maintain blood sugar levels. Schedule regular eye exams.",
        "Glaucoma": "Use prescribed eye drops. Monitor intraocular pressure regularly.",
        "Normal": "No signs of disease detected. Continue routine eye checkups."
    }

    advice = precautions.get(predicted_class, "Consult a specialist for further evaluation.")
    pdf.multi_cell(0, 10, txt=f"Precautionary Advice:\n{advice}")

    pdf_path = os.path.join(app.config['UPLOAD_FOLDER'], f"{filename}_report.pdf")
    pdf.output(pdf_path)
    return pdf_path

# --- Routes ---
@app.route('/')
def home():
    return render_template('index.html')

@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']
        hashed_pw = generate_password_hash(password, method='pbkdf2:sha256')

        if User.query.filter_by(username=username).first():
            flash('Username already exists')
```

```python
def register():
        if User.query.filter_by(username=username).first():
            flash('Username already exists')
            return redirect(url_for('register'))

        new_user = User(username=username, password=hashed_pw)
        db.session.add(new_user)
        db.session.commit()
        flash('Registration successful. Please log in.')
        return redirect(url_for('login'))

    return render_template('register.html')

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']
        user = User.query.filter_by(username=username).first()

        if user and check_password_hash(user.password, password):
            login_user(user)
            return redirect(url_for('detector'))
        else:
            flash('Invalid credentials')
            return redirect(url_for('login'))

    return render_template('login.html')
```

```python
@app.route('/logout')
@login_required
def logout():
    logout_user()
    flash('Logged out successfully.')
    return redirect(url_for('login'))

@app.route('/detector', methods=['GET', 'POST'])
@login_required
def detector():
    if request.method == 'POST':
        if 'file' not in request.files:
            flash('No file part')
            return redirect(request.url)
        file = request.files['file']

        if file.filename == '':
            flash('No selected file')
            return redirect(request.url)

        if file and allowed_file(file.filename):
            filename = secure_filename(file.filename)
            filepath = os.path.join(app.config['UPLOAD_FOLDER'], filename)
            image_bytes = file.read()
            with open(filepath, 'wb') as f:
                f.write(image_bytes)

            predicted_class, confidence = predict_disease(image_bytes)
            pdf_path = generate_pdf(predicted_class.replace('_', ' ').title(), confidence, filename)
```

```python
def detector():

        if file and allowed_file(file.filename):
            filename = secure_filename(file.filename)
            filepath = os.path.join(app.config['UPLOAD_FOLDER'], filename)
            image_bytes = file.read()
            with open(filepath, 'wb') as f:
                f.write(image_bytes)

            predicted_class, confidence = predict_disease(image_bytes)
            pdf_path = generate_pdf(predicted_class.replace('_', ' ').title(), confidence, filename)

            return render_template('result.html',
                                   predicted_class=predicted_class.replace('_', ' ').title(),
                                   confidence=confidence,
                                   image_url=url_for('static', filename='uploads/' + filename),
                                   pdf_url=url_for('static', filename='uploads/' + f"{filename}_report.pdf"))

    return render_template('detector.html')

# --- Main Execution ---
if __name__ == '__main__':
    with app.app_context():
        db.create_all()
    app.run(debug=True)
```

Deployment

Prerequisites

- GitHub repository with app.py, templates/, static/, model/my_model.h5, requirements.txt
- Render account
- Environment variable: SECRET_KEY set in Render dashboard
- Optional: Docker installed locally for reproducible builds

Quick deploy on Render

1. Push your repo to GitHub.
2. Create a new Web Service on Render.
3. Select your repo and set these:
    - Build Command: leave default (Render auto-installs from requirements.txt)
    - Start Command:

      gunicorn app:app --timeout 120
    - Environment:
    - Add SECRET_KEY = your_secret_key
4. Click Create. Wait until you see "Your service is live".
5. App will be available at your Render URL (e.g., https://eyeintel.onrender.com).

Local run

python -m venv venv

venv\Scripts\activate

pip install -r requirements.txt

python app.py

- App runs at http://127.0.0.1:5000